

The LISA global fit — how it is built

Four lines to run one. Three peer blocks to configure one. One container per walker to score one.

■ structure — what contains what ■ time — what happens when

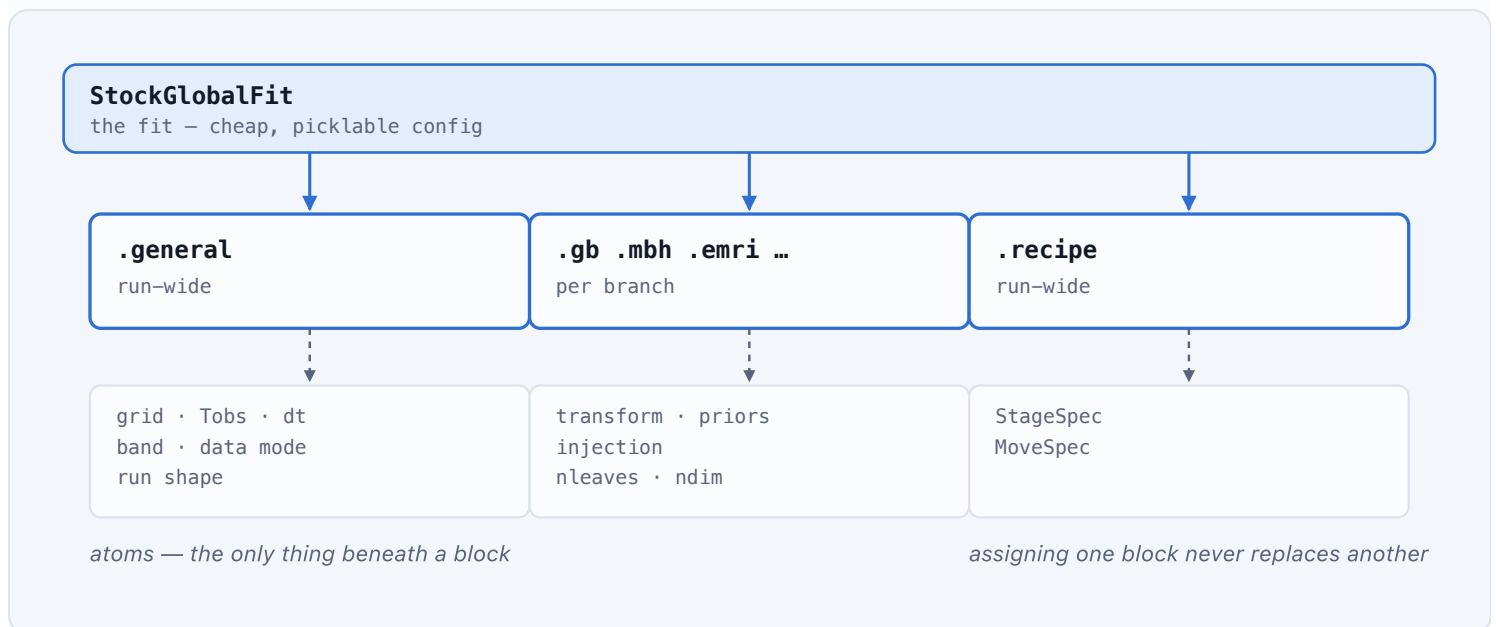
1 • Running one

```
from lisatools.globalfit.stock import erebor

fit = erebor.gb_no_fg(nwalkers=4)  # pick a stock fit
curr = fit.build()                 # heavy: data, domain, backend
fit.run()                         # sample
```

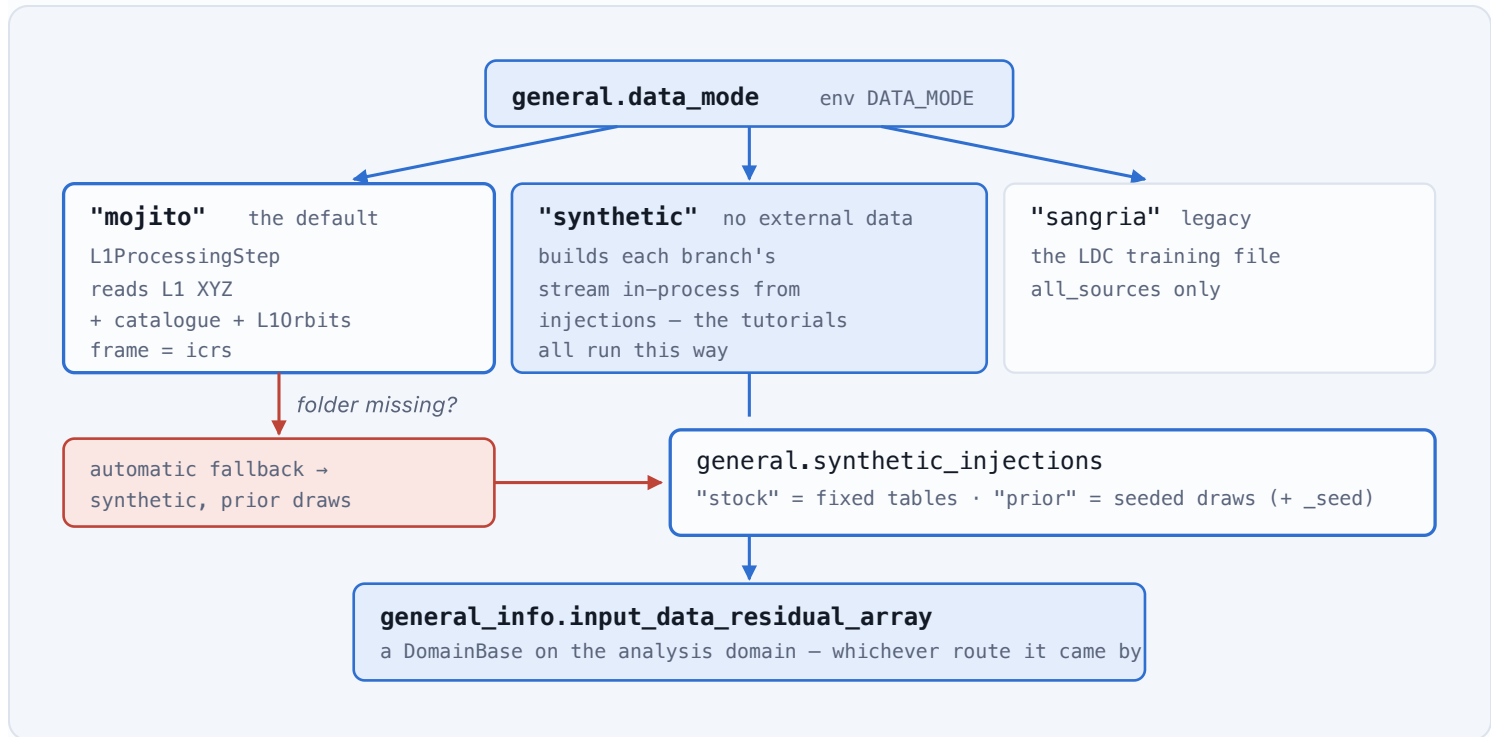
That is the whole surface. No settings file. Everything below is what those four lines work with.

2 • The shape: three peers



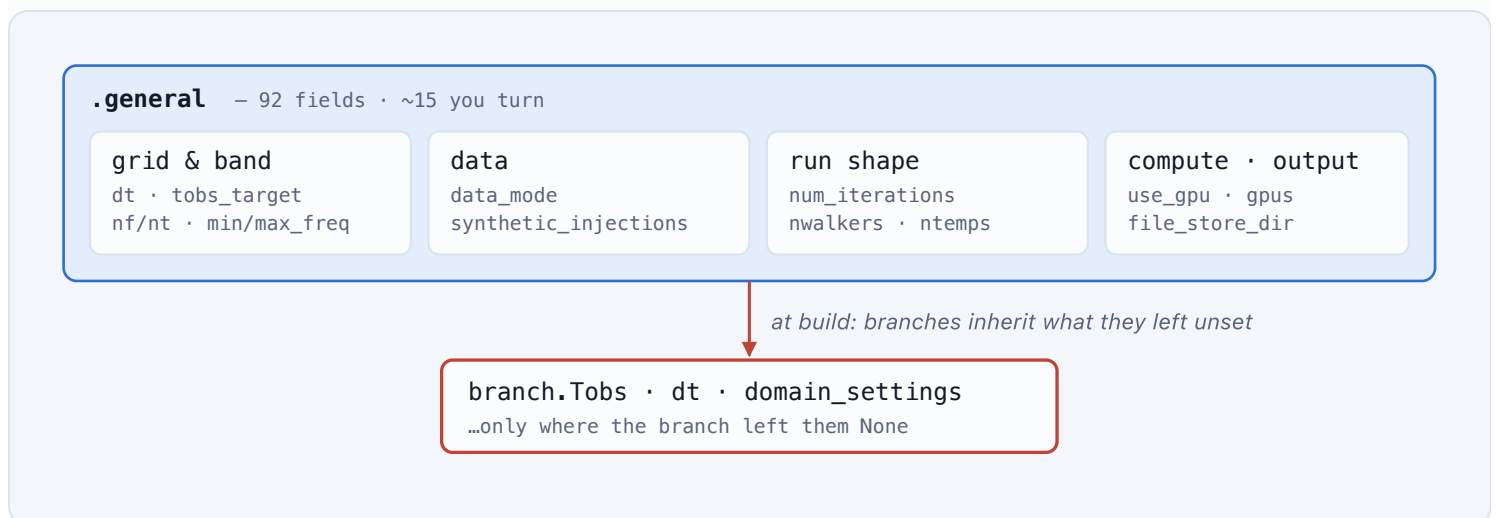
Peers, not a stack. `fit.general = ...` does not change `fit.gb` or `fit.recipe`.

3 • Data in — one knob swaps the whole pipeline



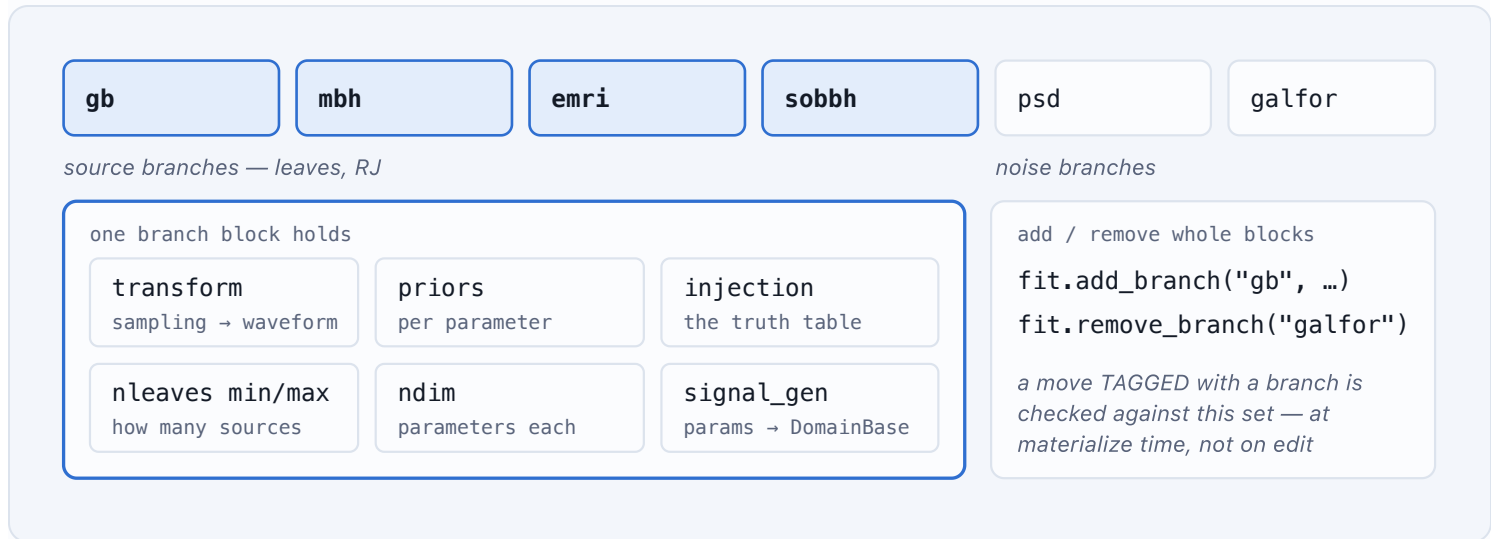
Every route lands in the same place. The processor is chosen at `build()`; downstream, nothing knows which one ran. Swapping `data_processor_class` outright always wins over the knob — it lets you adjust the source of the inputs without changing anything else downstream.

4 • general — the run-wide block



Run-wide scope, and a defaulting source. Its choices flow *down* into branches at build — defaulting, not ownership.

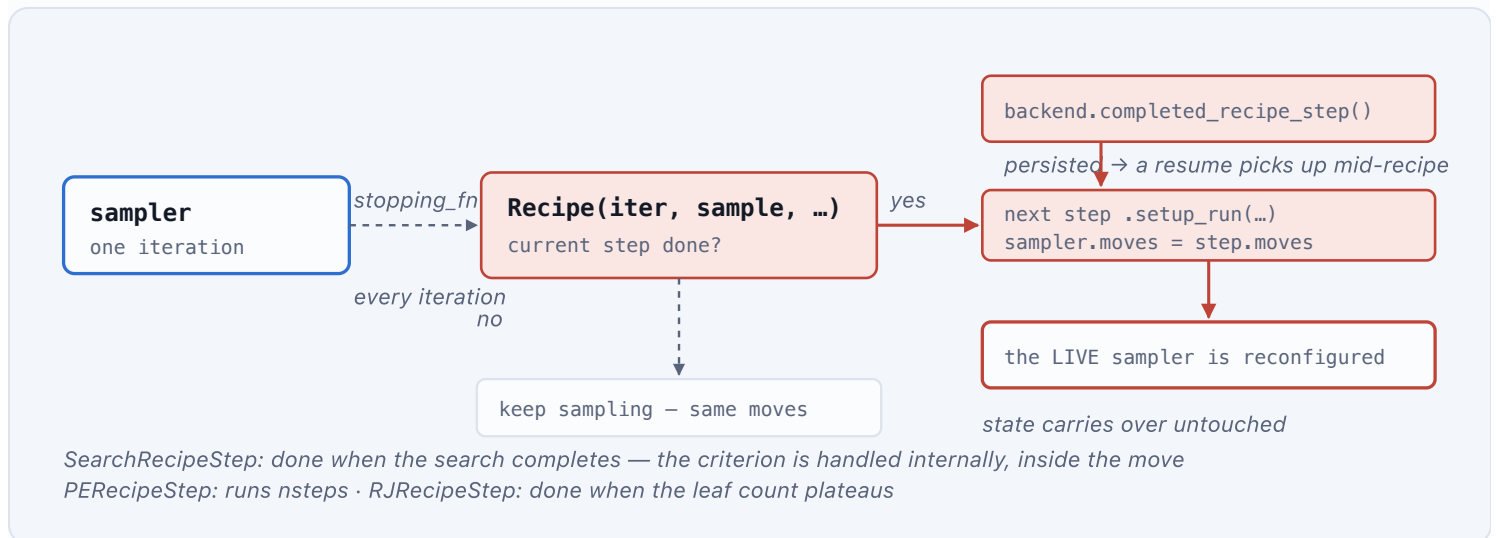
5 · branches — one block per source class



Per-source-class scope. Each block owns its parameter basis.

6 · recipe — a plan, not a move stack

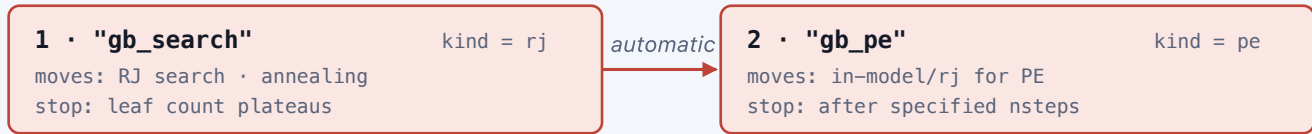
The purpose: change the sampler's setup *while it runs*. The recipe is handed to the sampler as its stopping function and consulted every iteration. When a stage is done, the next stage reconfigures the live sampler — the state, and everything the previous stage found, carries straight over.



Each stage answers two questions: how should the sampler be set up for me? (`setup_run`) and when am I done? (`stopping_function`).

6a · What you would use it for: search → PE, in-process

with a recipe — one run



the leaves the search found are already in the state — nothing is saved, reloaded or re-seeded

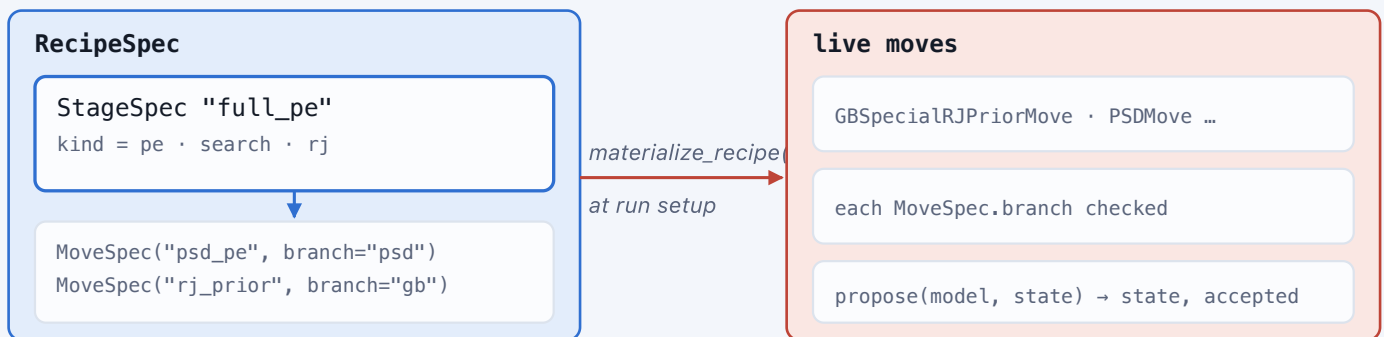
without one — four manual steps, outside the run



every hand-off is a chance to mis-seed the state — and none of it is recorded in the run

The recipe turns that four-step manual hand-off into two `StageSpec`s. The search stage's own stopping criterion decides when it has found what it can; the sampler is then reconfigured for PE in place, and the completed stage is written to the backend so a resume knows where it was.

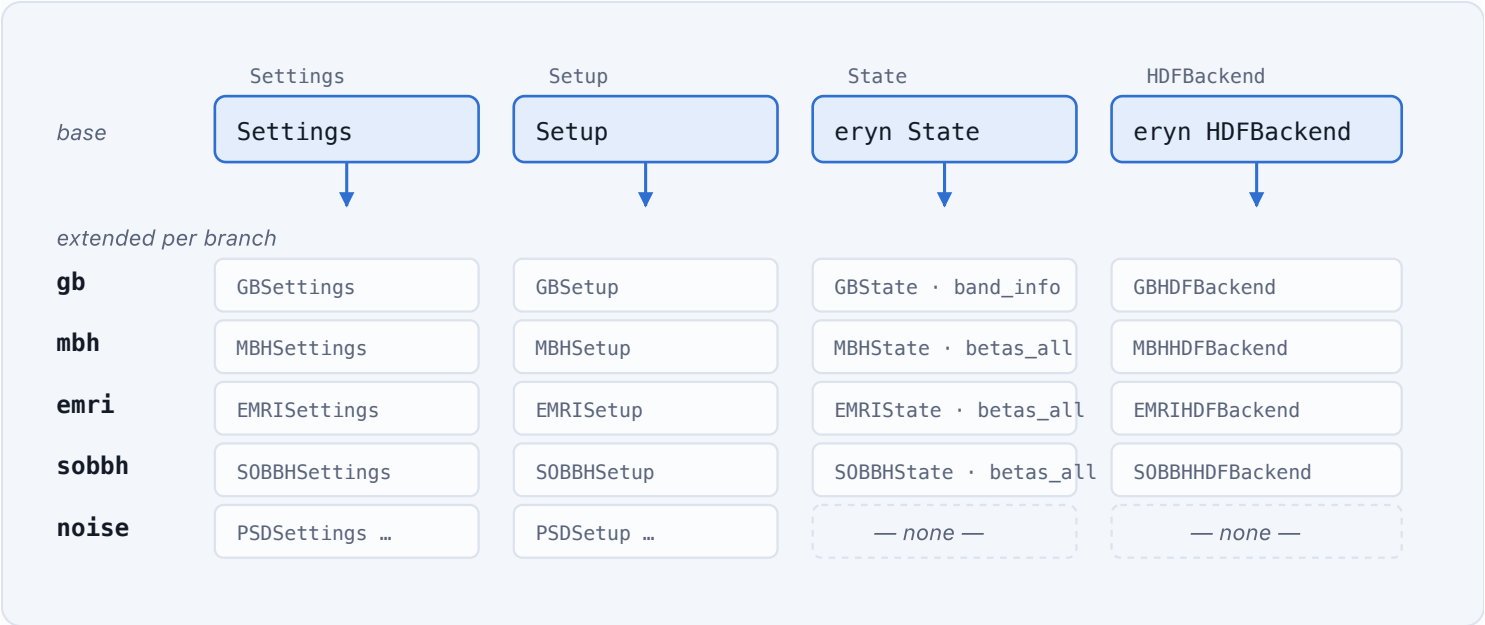
6b · How a stage is described



a Spec is a picklable description — like `importlib's ModuleSpec` — not the object

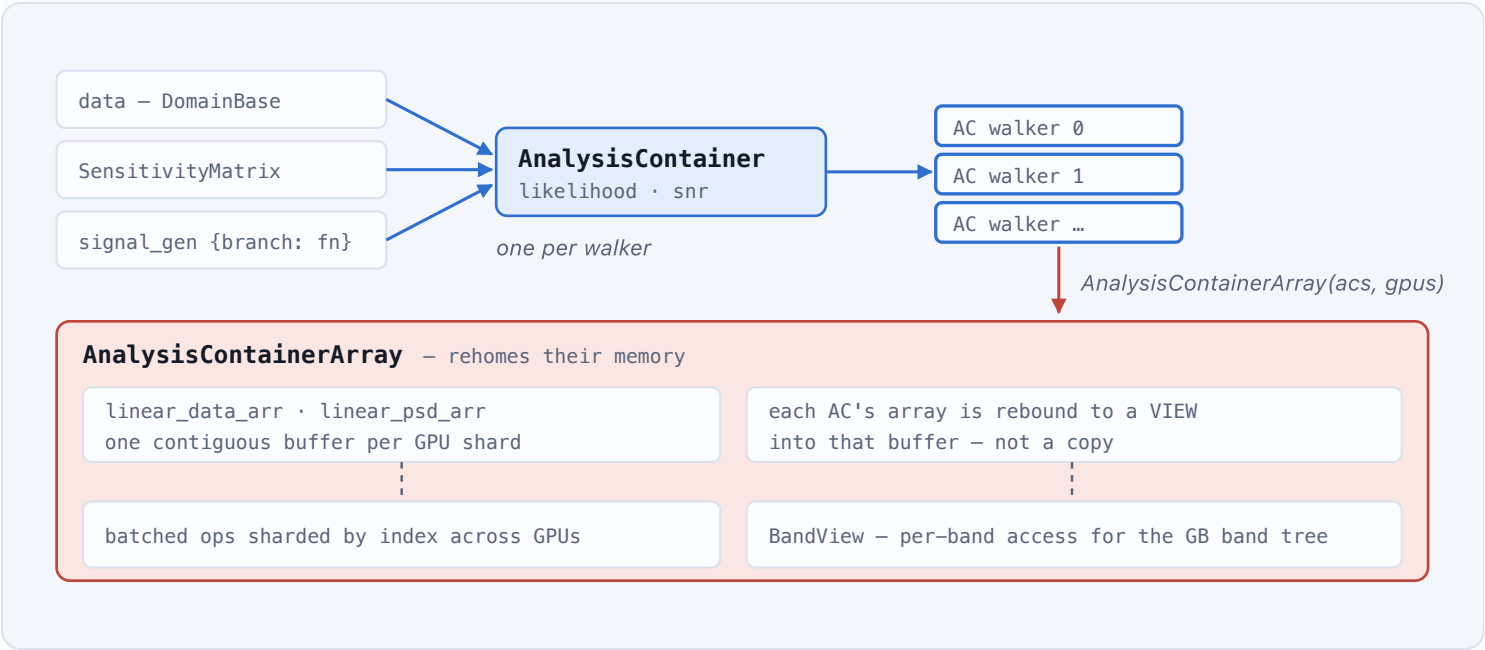
Declarative. The spec pickles and travels; the live move is constructed only inside the run's setup. Your own module is just a `MoveSpec` with a factory.

7 · Settings · Setup · State · HDFBackend



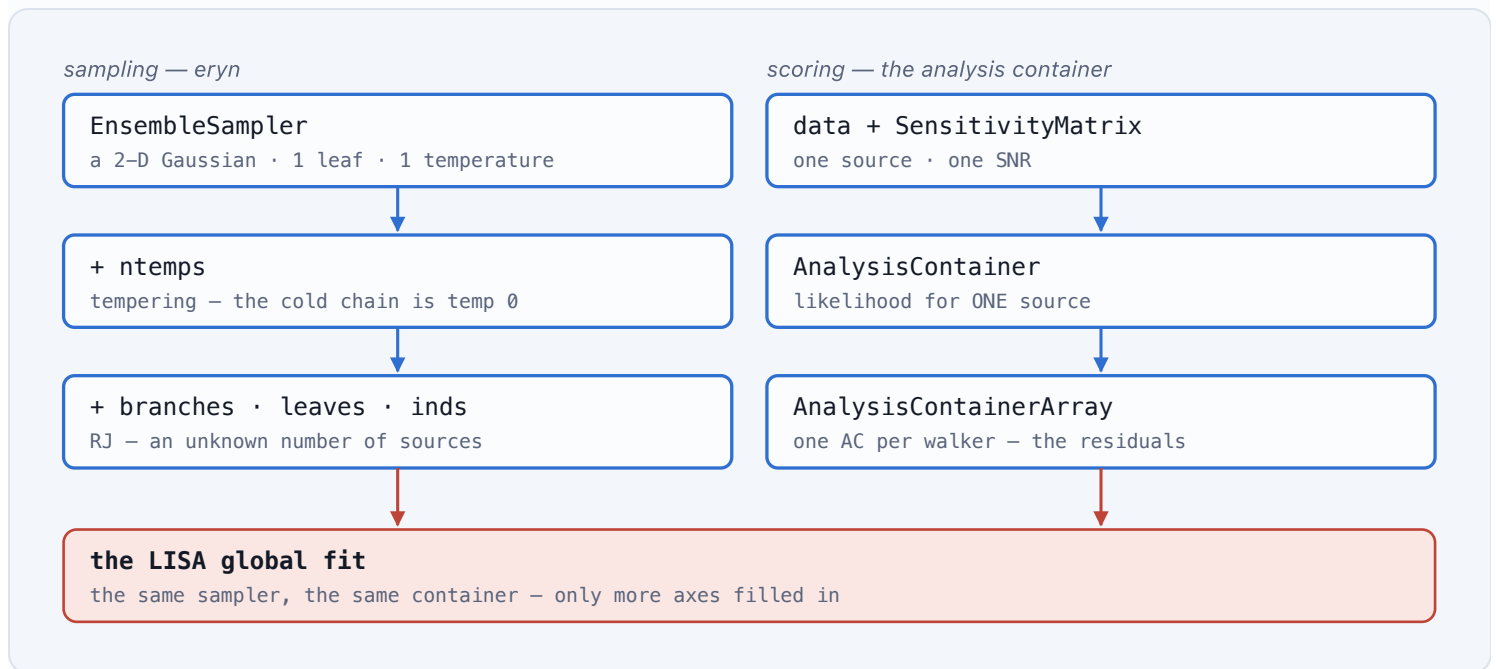
The same four bases, extended per branch. Source branches carry per-source bookkeeping — GB a per-band ladder, the rest a per-leaf one — so each needs a State to hold it and a backend to persist it. The noise branches (psd , galfor , sgwb) have none, so they stop at two. Each backend stores exactly what its State holds.

8 · AnalysisContainer → AnalysisContainerArray



The residual and the PSD live in the AnalysisContainerArray, not the containers. Each walker gets its own residual copy and its own sensitivity (re-evaluated from that walker's psd coords); the AnalysisContainerArray then copies them into one contiguous buffer per GPU and points every container at a slice of it. That rebinding is what makes batched, multi-GPU likelihoods possible.

8a · small → large — the same objects, scaled



Nothing new is introduced at the top. Both ladders end at the global fit because the global fit *is* the small case with more axes: the toy sampler already has the temperature and leaf axes, they are just length 1.

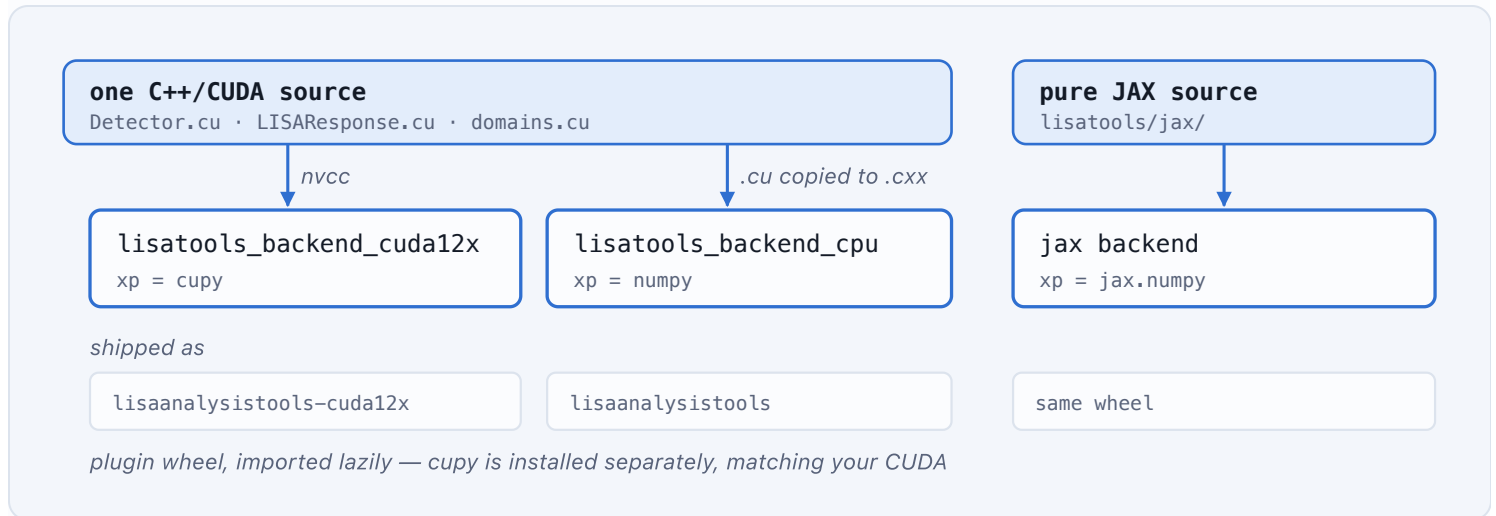
```
# one array — the same four axes the whole way up
#   state.branches_coords[branch] -> (ntemps, nwalkers, nleaves, ndim)

a toy Gaussian      ( 1, 32, 1, 2)
+ tempering         (10, 32, 1, 2)
+ RJ leaves         (10, 32, 100, 8) # inds[] turns leaves on/off
the global fit      (10, 32, 100, 8) # ...once per branch

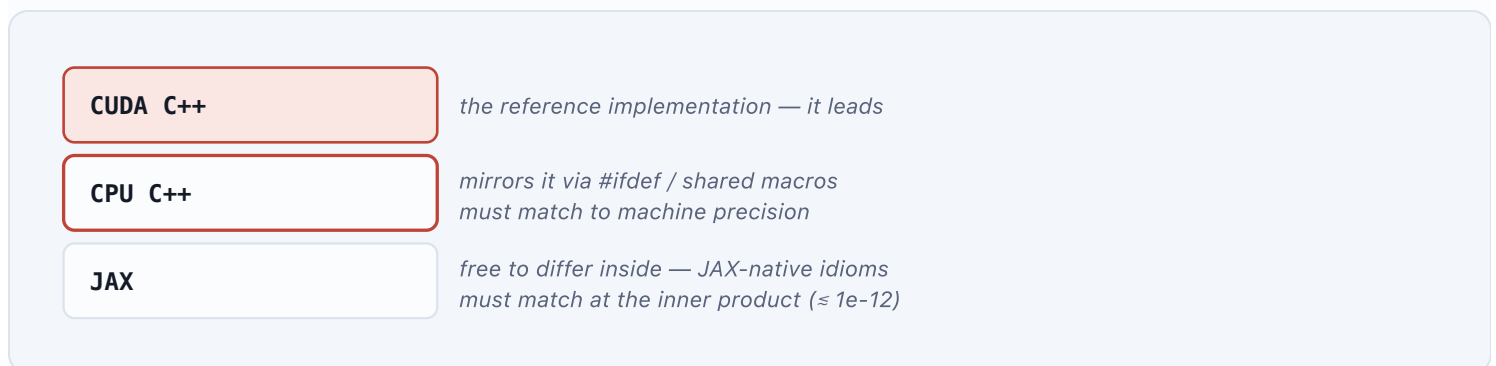
# and the scoring side
AnalysisContainer(data, sens, signal_gen).snr() # one source
AnalysisContainerArray([ac_0 ... ac_nwalkers]) # all walkers
```

Shapes are illustrative; the axes are the point. A toy chain and a production global fit read out of the same `(ntemps, nwalkers, nleaves, ndim)` block — which is why what you learn on a 2-D Gaussian transfers untouched, and why `gb`, `mbh`, `emri` ... are just more of that block.

8b · Backends — one source, three targets



The same `.cu` is compiled twice. `nvcc` builds it for the GPU; the build copies it to `.cxx` and the C++ compiler builds it for the CPU — so the two can never drift. CUDA ships as separate plugin wheels the main package imports lazily.

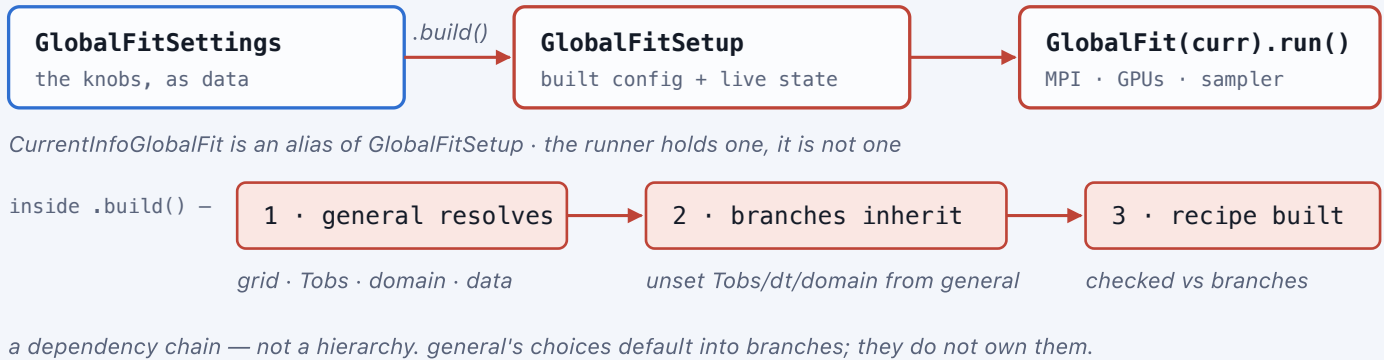


One reference, two mirrors. The hierarchy is what keeps three implementations honest: CPU is held to the GPU bit-for-bit, JAX only where it counts.



The backend is picked at construction — never as a per-call keyword. Every method dispatches on `self.backend`, so one object cannot be half CPU and half GPU. Python code stays array-module agnostic: resolve `xp` from the array, never `import cupy` unconditionally.

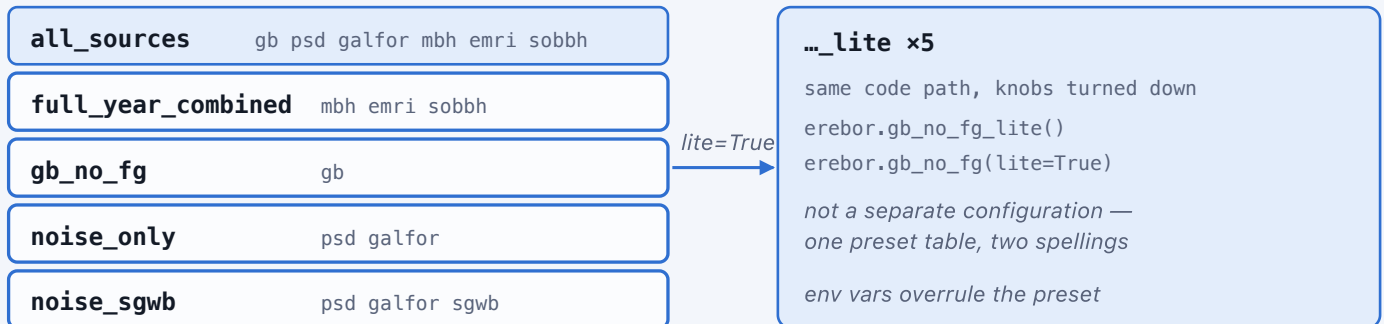
9 • Lifecycle and ordering at the highest level



Same `*Settings` → `*Setup` pattern as every module, one level up.

10 • The stock fits

five variants — branches they compose



Ten registered options; `erebor.get_stock_options()` lists them. A new variant is a subclass, never a settings file.